

High Performance Computing

Week 2 Task: Memory Locality and Parallel Program Design

The purpose of this task is to explore how memory access patterns can affect program performance and to understand the role of memory locality in High Performance Computing. You will compare row-wise and column-wise access to a matrix, identify spatial locality in a simple program, and apply the main steps of parallel program design: partitioning, communication, agglomeration, and mapping. No parallel programming library is required for this task.

Part 1: Memory Hierarchy

Answer the following questions briefly.

1. Arrange the following from fastest to slowest:

Main Memory, L1 Cache, Registers, Local Disk, L2 Cache

2. What is a **cache hit**?
3. What is a **cache miss**?
4. Explain briefly the difference between:
 - a) temporal locality;
 - b) spatial locality.
5. Which type of locality is mainly exploited when a program accesses consecutive elements of an array?

Part 2: Matrix Access Experiment

Consider a square matrix containing integers.

In C++, a two-dimensional matrix is normally stored in **row-major order**. Therefore, elements from the same row are stored next to each other in memory.

Create the following program.

```
1 #include <iostream>
2 #include <vector>
3 #include <chrono>
4
5 int main() {
6
7     const int N = 4000;
8
9     std::vector<int> A(N * N, 1);
10
11     long long sum = 0;
```

```

12
13     // Your code here
14
15     std::cout << "Sum = " << sum << std::endl;
16
17     return 0;
18 }

```

For the one-dimensional vector, access matrix element (i, j) as:

```

1 A[i * N + j]

```

Experiment A: Row-wise access

Calculate the sum of all matrix elements using:

```

1 for (int i = 0; i < N; ++i) {
2     for (int j = 0; j < N; ++j) {
3         sum += A[i * N + j];
4     }
5 }

```

Measure the execution time using `std::chrono`.

For example:

```

1 auto start = std::chrono::high_resolution_clock::now();
2
3 // code being measured
4
5 auto finish = std::chrono::high_resolution_clock::now();
6
7 std::chrono::duration<double> elapsed = finish - start;
8
9 std::cout << "Time = "
10           << elapsed.count()
11           << " seconds\n";

```

Run the program at least three times and record the execution times.

Run	Time (seconds)
1	
2	
3	

Experiment B: Column-wise access

Now change only the order of the loops:

```

1 for (int j = 0; j < N; ++j) {
2     for (int i = 0; i < N; ++i) {
3         sum += A[i * N + j];
4     }
5 }

```

Again run the program at least three times.

Run	Time (seconds)
1	
2	
3	

Questions

1. Do both versions calculate the same result?
2. Which version was faster on your computer?
3. In the row-wise version, are consecutive memory locations accessed frequently?
4. Which version has better spatial locality?
5. Explain briefly why changing only the order of two loops can affect execution time even though both programs perform the same number of additions.

Part 3: Simple Parallel Program Design

Consider the following problem.

We want to increase every element of a large matrix by one:

$$A_{i,j} \leftarrow A_{i,j} + 1.$$

Assume that the matrix has

$$4000 \times 4000$$

elements and that four processors are available:

$$P_0, P_1, P_2, P_3.$$

Apply the four basic steps of parallel program design.

1. Partitioning

Divide the matrix among the four processors.

A simple approach is to assign consecutive groups of rows to each processor.

Complete the table.

Processor	First Row	Last Row
P_0		
P_1		
P_2		
P_3		

Answer:

1. Can the four groups of rows be processed independently?
2. Is this an example of domain decomposition or functional decomposition?

2. Communication

Suppose every processor only performs:

$$A_{i,j} \leftarrow A_{i,j} + 1$$

for its own rows.

1. Does P_0 need data from P_1 during this computation?

2. Do neighboring processors need to exchange matrix elements?
3. Would you describe this computation as:
 - no communication;
 - local communication;
 - global communication?

3. Agglomeration

Suppose initially one task was created for every matrix element.

For a 4000×4000 matrix this would create

$$4000 \times 4000 = 16,000,000$$

very small tasks.

Instead, we can combine many elements into one larger task.

Answer:

1. Would one task per matrix element be fine-grained or coarse-grained parallelism?
2. Why might combining many small tasks into larger tasks improve efficiency?
3. In our four-processor example, suggest a reasonable larger task size.

4. Mapping

Assume that the four processors have the same performance.

Assign the four groups of matrix rows to processors.

Task	Processor
Rows 0–	
Rows –	
Rows –	
Rows –3999	

Answer:

1. Is the amount of work approximately balanced?
2. What could happen if one processor were assigned 3000 rows while each of the remaining processors received only about 333 rows?

Part 4: Reduction

Assume each processor calculates the sum of its own part of the matrix.

After computation we have:

$$S_0, \quad S_1, \quad S_2, \quad S_3.$$

Write an expression for the final matrix sum S .

$$S =$$

Answer the following:

1. Is combining S_0, S_1, S_2, S_3 a local or global operation?
2. What is this type of operation commonly called in parallel computing?

Part 5: Short Analysis

Consider the following two programs.

Program A

```
1 for (int i = 0; i < N; ++i)
2     for (int j = 0; j < N; ++j)
3         A[i * N + j]++;
```

Program B

```
1 for (int j = 0; j < N; ++j)
2     for (int i = 0; i < N; ++i)
3         A[i * N + j]++;
```

Answer:

1. Do the programs have the same asymptotic time complexity?
2. What is their time complexity using Big-O notation?
3. Even if their Big-O complexity is the same, can their actual execution times be different?
4. What hardware feature discussed in this week's lecture can explain the difference?

Submission

Submit:

1. the C++ source code;
2. execution times for row-wise and column-wise access;
3. completed tables;
4. answers to all questions.